МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему
«Вдосконалення структури коду графічного редактора об’єктів»

Виконав:
студент групи ІМ-55
Стеблевець Олександр Олександрович
Номер в списку – 26

Перевірив:
Рекечинський Д.О.

Київ 2026

Завдання

1. Створити у середовищі розробки проєкт з ім’ям Lab4 (подальший розвиток проєкту графічного редактора, розробленого у лабораторній роботі №3).
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Завдання згідно варіанту

Номер варіанту дорівнює номеру студента в журналі: Ж = 26.

1. Студенти з парним номером програмують **динамічний** об’єкт класу MyEditor, забезпечивши коректне його створення та знищення. Ж = 26 – парне, тому в контролері редактора (клас `EditorController`, що виконує роль MyEditor; назву обрано ідіоматичною для Flutter) об’єкт стану створюється в `initState` і явно знищується у `dispose`.
2. Усі кольори та стилі геометричних форм (за винятком «гумового» сліду) – як у лабораторній роботі №3: прямокутник вводиться від центру до кута з рожевим заповненням і чорним контуром, еліпс – по протилежних кутах охоплюючого прямокутника без заповнення. «Гумовий» слід при вводі **усіх** фігур малюється пунктирною лінією.
3. Окрім чотирьох типів фігур з лабораторних робіт №2–3, запрограмовано введення та відображення двох нових фігур – **лінії з кружечками** та **каркасу куба**. Кольори ліній та заповнення цих фігур обрано на власний розсуд (чорний контур без заповнення).

4. Класи нових фігур має бути запрограмовано множинним успадкуванням. Мова Dart множинного успадкування не підтримує, тому, як дозволяє завдання, його замінено **композицією-делегуванням**: клас `CircleLineShape` містить об'єкти `LineShape` і два `EllipseShape` і для відображення викликає їхні методи `paint()`; клас `CubeShape` містить два `RectangleShape` (грані) і чотири `LineShape` (ребра) – теж через виклики `paint()`. Пряма вимога завдання «показувати лінії методом Show з класу LineShape, кружечки – з класу EllipseShape, прямокутники – з класу RectShape» виконується буквально: нові класи самі нічого не малюють.
5. Для всіх шести типів форм реалізовано кнопки панелі Toolbar з підказками (tooltips). Позначка поточного типу об'єкта, як і в роботі №3, – у заголовку вікна та підсвічуванням кнопки.
6. У звіті наведена схема успадкування класів – діаграма класів.

Лістинги програми

```dart
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';

import 'editor/editor_page.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await windowManager.ensureInitialized();
  runApp(const PaintApp());
}

class PaintApp extends StatelessWidget {
  const PaintApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Paint',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const EditorPage(title: 'Paint'),
    );
  }
}
```

Лістинг 1 – main.dart

```dart
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';
```

```dart
import 'editor_controller.dart';
import 'editor_menu.dart';
import 'editor_toolbar.dart';
import 'paint_canvas.dart';
import 'shape/shape_type.dart';

class EditorPage extends StatefulWidget {
  const EditorPage({super.key, required this.title});

  final String title;

  @override
  State<EditorPage> createState() => _EditorPageState();
}

class _EditorPageState extends State<EditorPage> {
  late final EditorController _editor;

  @override
  void initState() {
    super.initState();
    _editor = EditorController()..addListener(_handleEditorChanged);
    _updateTitle(_editor.selectedTool);
  }

  @override
  void dispose() {
    _editor
      ..removeListener(_handleEditorChanged)
      ..destroy();
    super.dispose();
  }

  void _handleEditorChanged() {
    setState(() {});
  }

  void _selectTool(ShapeType? tool) {
    _editor.selectTool(tool);
    _updateTitle(tool);
  }

  void _updateTitle(ShapeType? tool) {
    windowManager.setTitle(tool == null ? 'Paint' : 'Paint (${tool.label})');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // appBar: AppBar(title: Text(widget.title)),
      body: Column(
        children: [
          Align(
            alignment: AlignmentDirectional.centerStart,
            child: EditorMenu(selectedTool: _editor.selectedTool, onToolSelected: _selectTool),
          ),
          EditorToolbar(selectedTool: _editor.selectedTool, onToolSelected: _selectTool),
          Expanded(child: PaintCanvas(editor: _editor)),
        ],
      ),
    );
  }
}
```

Лістинг 2 – editor_page.dart

```dart
import 'package:flutter/foundation.dart';

import 'canvas_shapes.dart';
import 'shape/shape.dart';
import 'shape/shape_type.dart';

class EditorController extends ChangeNotifier {
  CanvasShapes? _shapes = CanvasShapes();
  ShapeType? _selectedTool;
```

```
  bool get isDestroyed => _shapes == null;

  CanvasShapes get shapes {
    final storage = _shapes;
    if (storage == null) {
      throw StateError('EditorController is destroyed');
    }
    return storage;
  }

  ShapeType? get selectedTool => _selectedTool;

  void selectTool(ShapeType? tool) {
    if (_selectedTool == tool) {
      return;
    }
    _selectedTool = tool;
    notifyListeners();
  }

  void addShape(Shape shape) => shapes.add(shape);

  void destroy() => dispose();

  @override
  void dispose() {
    if (isDestroyed) {
      return;
    }
    _shapes!.shapes.clear();
    _shapes = null;
    _selectedTool = null;
    super.dispose();
  }
}
```

Лістинг 3 – editor_controller.dart

```
import 'package:flutter/material.dart';
import 'package:paint/editor/shape/shape_type.dart';

class EditorMenu extends StatelessWidget {
  const EditorMenu({
    super.key,
    required this.selectedTool,
    required this.onToolSelected,
  });

  final ShapeType? selectedTool;
  final ValueChanged<ShapeType?> onToolSelected;

  @override
  Widget build(BuildContext context) {
    return MenuBar(
      children: [
        SubmenuButton(
          menuChildren: const [],
          child: const Text('Файл'),
        ),
        SubmenuButton(
          menuChildren: [
            for (final ShapeType tool in ShapeType.values)
              CheckboxMenuButton(
                value: selectedTool == tool,
                onChanged: (bool? selected) => onToolSelected(selected == true ? tool : null),
                child: Text(tool.label),
              ),
          ],
          child: const Text('Об\'єкти'),
        ),
        SubmenuButton(
          menuChildren: const [],
          child: const Text('Довідка'),
        ),
      ],
```

```
    );
  }
}
```

Лістинг 4 – editor_menu.dart

```dart
import 'package:flutter/material.dart';

import 'shape/shape_type.dart';

class EditorToolbar extends StatelessWidget {
  const EditorToolbar({
    super.key,
    required this.selectedTool,
    required this.onToolSelected,
  });

  final ShapeType? selectedTool;
  final ValueChanged<ShapeType?> onToolSelected;

  Widget _buildButton(ColorScheme colorScheme, ShapeType tool) {
    final selected = selectedTool == tool;
    return IconButton(
      icon: Icon(tool.icon),
      tooltip: tool.label,
      isSelected: selected,
      onPressed: () => onToolSelected(selected ? null : tool),
      style: ButtonStyle(
        backgroundColor: WidgetStatePropertyAll<Color?>(
          selected ? colorScheme.secondaryContainer : null,
        ),
        foregroundColor: WidgetStatePropertyAll<Color?>(
          selected ? colorScheme.onSecondaryContainer : null,
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Material(
      color: colorScheme.surfaceContainer,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 2),
        child: Row(
          children: [
            for (final ShapeType tool in ShapeType.values) _buildButton(colorScheme, tool),
          ],
        ),
      ),
    );
  }
}
```

Лістинг 5 – editor_toolbar.dart

```dart
import 'package:flutter/material.dart';

import 'canvas_shapes.dart';
import 'editor_controller.dart';
import 'shape/shape.dart';
import 'shape/shape_type.dart';

class PaintCanvas extends StatefulWidget {
  const PaintCanvas({super.key, required this.editor});

  final EditorController editor;

  @override
  State<PaintCanvas> createState() => _PaintCanvasState();
}

class _PaintCanvasState extends State<PaintCanvas> {
  Offset? _dragStart;
  Offset? _dragCurrent;
```

```
  Shape _buildShape(Offset start, Offset end) =>
      Shape.of(widget.editor.selectedTool!, start: start, end: end);

  bool get _isDragTool {
    final tool = widget.editor.selectedTool;
    return tool != null && tool.usesDrag;
  }

  void _handleTapDown(TapDownDetails details) {
    final tool = widget.editor.selectedTool;
    if (tool == null || tool.usesDrag) {
      return;
    }
    final position = details.localPosition;
    widget.editor.addShape(_buildShape(position, position));
  }

  void _handlePanStart(DragStartDetails details) {
    if (!_isDragTool) {
      return;
    }
    setState(() {
      _dragStart = details.localPosition;
      _dragCurrent = details.localPosition;
    });
  }

  void _handlePanUpdate(DragUpdateDetails details) {
    if (!_isDragTool) {
      return;
    }
    setState(() {
      _dragCurrent = details.localPosition;
    });
  }

  void _handlePanEnd(DragEndDetails details) {
    if (!_isDragTool) {
      return;
    }
    final start = _dragStart;
    final current = _dragCurrent;
    setState(() {
      _dragStart = null;
      _dragCurrent = null;
    });
    if (start == null || current == null) {
      return;
    }
    widget.editor.addShape(_buildShape(start, current));
  }

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTapDown: _handleTapDown,
      onPanStart: _handlePanStart,
      onPanUpdate: _handlePanUpdate,
      onPanEnd: _handlePanEnd,
      child: ColoredBox(
        color: Colors.white,
        child: CustomPaint(
          painter: _CanvasPainter(
            storage: widget.editor.shapes,
            tool: widget.editor.selectedTool,
            dragStart: _dragStart,
            dragCurrent: _dragCurrent,
          ),
          child: const SizedBox.expand(),
        ),
      ),
    );
  }
}
```

```dart
class _CanvasPainter extends CustomPainter {
  const _CanvasPainter({
    required CanvasShapes storage,
    required this.tool,
    required this.dragStart,
    required this.dragCurrent,
  }) : _storage = storage,
       super(repaint: storage);

  final CanvasShapes _storage;
  final ShapeType? tool;
  final Offset? dragStart;
  final Offset? dragCurrent;

  @override
  void paint(Canvas canvas, Size size) {
    canvas.clipRect(Offset.zero & size);
    for (final shape in _storage.shapes) {
      shape.paint(canvas);
    }
    final start = dragStart;
    final current = dragCurrent;
    if (start == null || current == null || tool == null) {
      return;
    }
    Shape.of(tool!, start: start, end: current).paintPreview(canvas);
  }

  @override
  bool shouldRepaint(covariant _CanvasPainter oldDelegate) =>
      oldDelegate.dragStart != dragStart || oldDelegate.dragCurrent != dragCurrent;
}
```

Лістинг 6 – paint_canvas.dart

```dart
import 'package:flutter/foundation.dart';

import 'shape/shape.dart';

class CanvasShapes extends ChangeNotifier {
  final List<Shape> shapes = <Shape>[];

  int get count => shapes.length;

  void add(Shape shape) {
    shapes.add(shape);
    notifyListeners();
  }
}
```

Лістинг 7 – canvas_shapes.dart

```dart
import 'package:flutter/material.dart';

enum ShapeType {
  dot('Крапка', icon: Icons.trip_origin, usesDrag: false),
  line('Лінія', icon: Icons.horizontal_rule),
  rectangle('Прямокутник', icon: Icons.crop_square),
  ellipse('Еліпс', icon: Icons.panorama_fish_eye),
  circleLine('Лінія з кружечками', icon: Icons.timeline),
  cube('Каркас куба', icon: Icons.view_in_ar);

  const ShapeType(this.label, {required this.icon, this.usesDrag = true});

  final String label;

  final IconData icon;

  final bool usesDrag;
}
```

Лістинг 8 – shape_type.dart

```dart
import 'dart:math' as math;
```

```dart
import 'package:flutter/material.dart';

import 'circle_line_shape.dart';
import 'cube_shape.dart';
import 'dot_shape.dart';
import 'ellipse_shape.dart';
import 'line_shape.dart';
import 'rectangle_shape.dart';
import 'shape_type.dart';

abstract class Shape {
  const Shape({required this.start, required this.end});

  static const double _dash = 6;
  static const double _gap = 4;

  final Offset start;
  final Offset end;

  factory Shape.of(ShapeType type, {required Offset start, Offset? end}) =>
      switch (type) {
        ShapeType.dot => DotShape(start: start),
        ShapeType.line => LineShape(start: start, end: end ?? start),
        ShapeType.rectangle => RectangleShape(start: start, end: end ?? start),
        ShapeType.ellipse => EllipseShape(start: start, end: end ?? start),
        ShapeType.circleLine => CircleLineShape(start: start, end: end ?? start),
        ShapeType.cube => CubeShape(start: start, end: end ?? start),
      };

  void drawDashed(Canvas canvas, Paint paint, Path path) {
    for (final metric in path.computeMetrics()) {
      double distance = 0;
      while (distance < metric.length) {
        final next = math.min(distance + _dash, metric.length);
        canvas.drawPath(metric.extractPath(distance, next), paint);
        distance = next + _gap;
      }
    }
  }

  void paint(Canvas canvas);

  void paintPreview(Canvas canvas);
}
```

Лістинг 9 – shape.dart

```dart
import 'package:flutter/material.dart';

import 'shape.dart';

class DotShape extends Shape {
  const DotShape({required super.start}) : super(end: start);

  @override
  void paint(Canvas canvas) {
    canvas.drawCircle(start, 3, Paint()..color = Colors.black);
  }

  @override
  void paintPreview(Canvas canvas) {}
}
```

Лістинг 10 – dot_shape.dart

```dart
import 'package:flutter/material.dart';

import 'shape.dart';

class LineShape extends Shape {
  const LineShape({required super.start, required super.end});

  @override
  void paint(Canvas canvas) {
    canvas.drawLine(start, end, _stroke(Colors.black));
  }
```

```
    @override
    void paintPreview(Canvas canvas) {
      drawDashed(
        canvas,
        _stroke(Colors.black),
        Path()
          ..moveTo(start.dx, start.dy)
          ..lineTo(end.dx, end.dy),
      );
    }

    static Paint _stroke(Color color) =>
        Paint()
          ..color = color
          ..style = PaintingStyle.stroke
          ..strokeWidth = 2;
}
```

Лістинг 11 – line_shape.dart

```
import 'package:flutter/material.dart';

import 'shape.dart';

class RectangleShape extends Shape {
  const RectangleShape({
    required super.start,
    required super.end,
    this.filled = true,
  });

  final bool filled;

  Rect get _rect => Rect.fromCenter(
    center: start,
    width: (end.dx - start.dx).abs() * 2,
    height: (end.dy - start.dy).abs() * 2,
  );

  Paint get _fill => Paint()..color = Colors.pink;

  Paint _stroke(Color color) =>
      Paint()
        ..color = color
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;

  @override
  void paint(Canvas canvas) {
    if (filled) {
      canvas.drawRect(_rect, _fill);
    }
    canvas.drawRect(_rect, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(canvas, _stroke(Colors.black), Path()..addRect(_rect));
  }
}
```

Лістинг 12 – rectangle_shape.dart

```
import 'package:flutter/material.dart';

import 'shape.dart';

class EllipseShape extends Shape {
  const EllipseShape({required super.start, required super.end});

  Rect get _rect => Rect.fromPoints(start, end);

  Paint _stroke(Color color) =>
      Paint()
        ..color = color
```

```
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;

  @override
  void paint(Canvas canvas) {
    canvas.drawOval(_rect, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(canvas, _stroke(Colors.black), Path()..addOval(_rect));
  }
}
```

Лістинг 13 – ellipse_shape.dart

```
import 'package:flutter/material.dart';

import 'ellipse_shape.dart';
import 'line_shape.dart';
import 'shape.dart';

class CircleLineShape extends Shape {
  CircleLineShape({required super.start, required super.end})
    : _line = LineShape(start: start, end: end),
      _startDot = _circle(start),
      _endDot = _circle(end);

  static const double _radius = 5;

  final LineShape _line;
  final EllipseShape _startDot;
  final EllipseShape _endDot;

  static EllipseShape _circle(Offset center) => EllipseShape(
    start: Offset(center.dx - _radius, center.dy - _radius),
    end: Offset(center.dx + _radius, center.dy + _radius),
  );

  @override
  void paint(Canvas canvas) {
    _line.paint(canvas);
    _startDot.paint(canvas);
    _endDot.paint(canvas);
  }

  @override
  void paintPreview(Canvas canvas) {
    final paint =
        Paint()
          ..color = Colors.black
          ..style = PaintingStyle.stroke
          ..strokeWidth = 2;
    drawDashed(
      canvas,
      paint,
      Path()
        ..moveTo(start.dx, start.dy)
        ..lineTo(end.dx, end.dy)
        ..addOval(Rect.fromCircle(center: start, radius: _radius))
        ..addOval(Rect.fromCircle(center: end, radius: _radius)),
    );
  }
}
```

Лістинг 14 – circle_line_shape.dart

```
import 'package:flutter/material.dart';

import 'line_shape.dart';
import 'rectangle_shape.dart';
import 'shape.dart';

class CubeShape extends Shape {
  CubeShape({required Offset start, required Offset end})
    : this._rects(Rect.fromPoints(start, end));
```

```
  CubeShape._rects(Rect front)
    : _front = front,
      _back = front.shift(_depth(front)),
      _frontFace = RectangleShape(
        start: front.center,
        end: front.bottomRight,
        filled: false,
      ),
      _backFace = RectangleShape(
        start: _rectAt(front, _depth(front)).center,
        end: _rectAt(front, _depth(front)).bottomRight,
        filled: false,
      ),
      _edges = [
        for (final corner in _corners(front))
          LineShape(start: corner, end: corner + _depth(front)),
      ],
      super(start: front.topLeft, end: front.bottomRight);

  static const double _depthFactor = 0.35;

  final Rect _front;
  final Rect _back;
  final RectangleShape _frontFace;
  final RectangleShape _backFace;
  final List<LineShape> _edges;

  static Offset _depth(Rect rect) =>
      Offset(rect.width * _depthFactor, -rect.height * _depthFactor);

  static Rect _rectAt(Rect rect, Offset shift) => Rect.fromLTWH(
    rect.left + shift.dx,
    rect.top + shift.dy,
    rect.width,
    rect.height,
  );

  static List<Offset> _corners(Rect rect) =>
      [rect.topLeft, rect.topRight, rect.bottomLeft, rect.bottomRight];

  @override
  void paint(Canvas canvas) {
    _backFace.paint(canvas);
    for (final edge in _edges) {
      edge.paint(canvas);
    }
    _frontFace.paint(canvas);
  }

  @override
  void paintPreview(Canvas canvas) {
    final paint =
        Paint()
          ..color = Colors.black
          ..style = PaintingStyle.stroke
          ..strokeWidth = 2;
    final path = Path()
      ..addRect(_front)
      ..addRect(_back);
    final frontCorners = _corners(_front);
    final backCorners = _corners(_back);
    for (var i = 0; i < frontCorners.length; i++) {
      path
        ..moveTo(frontCorners[i].dx, frontCorners[i].dy)
        ..lineTo(backCorners[i].dx, backCorners[i].dy);
    }
    drawDashed(canvas, paint, path);
  }
}
```

Лістинг 15 – cube_shape.dart

Діаграма

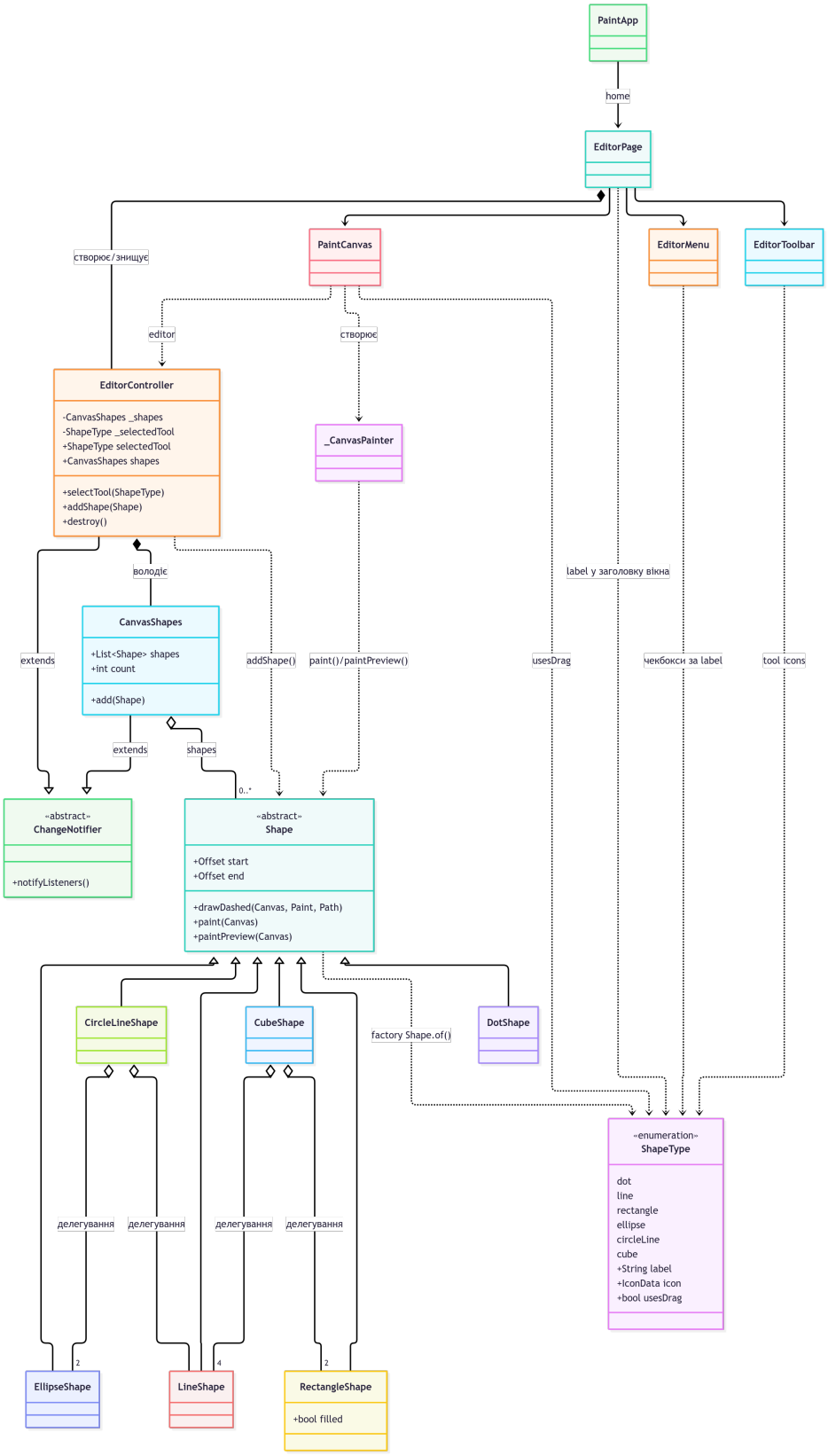


Рисунок 1 – Діаграма класів графічного редактора (успадкування, агрегація та делегування)

Скріншоти

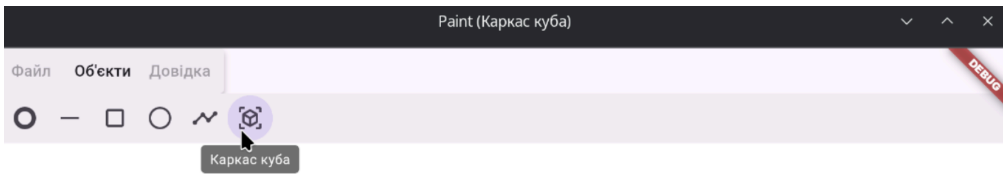


Рисунок 2 – Панель інструментів з кнопками для всіх шести типів фігур: вибір інструмента «Каркас куба» – підсвічена кнопка, підказка (tooltip) та позначка поточного типу об’єкта у заголовку вікна

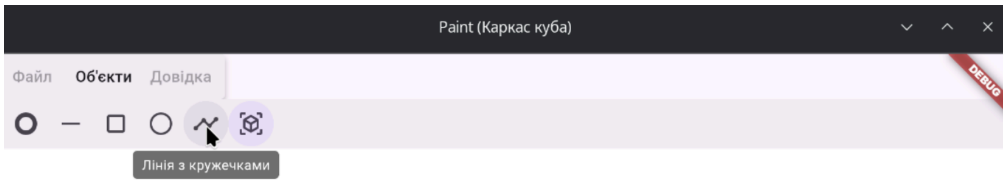


Рисунок 3 – Кнопка нової фігури «Лінія з кружечками» з підказкою

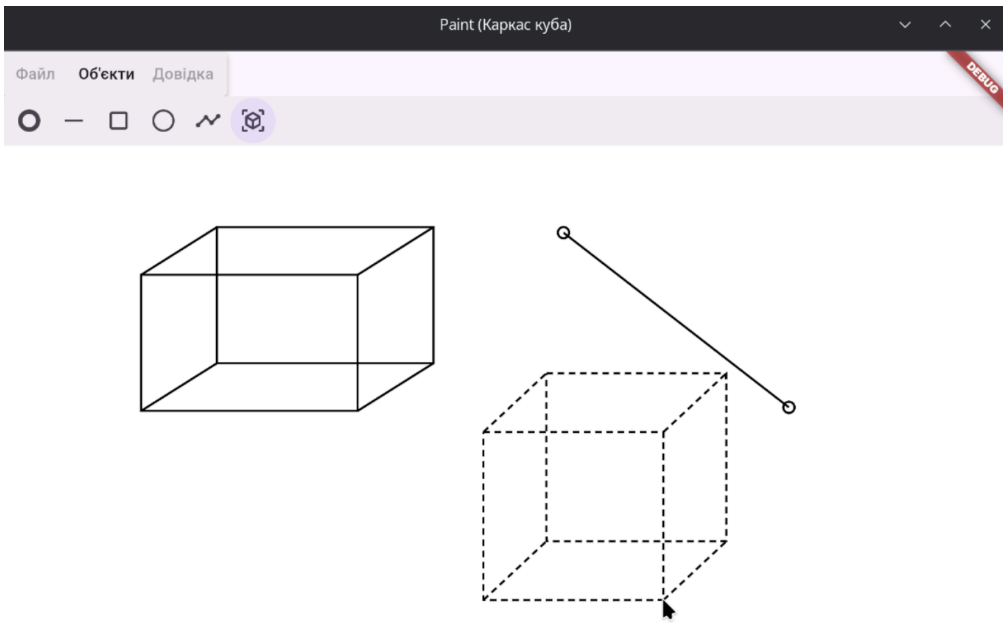


Рисунок 4 – Результат роботи: намальований каркас куба та лінія з кружечками на кінцях; пунктирний «гумовий» слід введення наступного куба

Висновки

Отримав досвід роботи з enum і конструкторами в Dart. Переніс всю фігурну логіку (як приклад - іконки) в сам enum. Також додав дві фігури в програму і переробив логіку внутрішнього стану редактора.